ERES Resonance Check — Workflow Failure Report

Issue Type: Infrastructure / CI Diagnostics
Severity: Non-blocking (NPR Level 1: Acknowledgment & Dialogue)
Filed by: Joseph A. Sprute (ERES Maestro)
Date: March 28, 2026
Workflow: `.github/workflows/ci.yml` → Job 3: `resonance`
Status: 🔴 FAILURE → 🟡 DIAGNOSED → 🟢 SIMULATOR PROPOSED

Summary

The ERES Resonance Check (Job 3 of ERES-CI workflow) fails during the **ERES framework coverage report** step. The locked-acronym consistency check (Step 1) passes, but the framework coverage report (Step 2) fails due to physical infrastructure incompatibility between the bash associative array implementation and the GitHub Actions runner environment.

This is a system failure, not a contributor failure. NPR applies.

Root Cause Analysis

Problem 1: Bash Associative Array Portability

The framework coverage report uses `declare -A FRAMEWORKS` with bash associative arrays:

```bash
declare -A FRAMEWORKS
FRAMEWORKS=(
  ["C=R×P/M"]="Foundational Equation"
  ["GraceChain\|Gracechain"]="Ethical Transaction Ledger"
  # ...
)
```

Failure modes:

- `declare -A` requires **bash 4.0+**. GitHub Actions `ubuntu-latest` ships bash 5.x, but the default `shell:` for `run:` steps may invoke `/bin/sh` (dash) depending on runner configuration, which does not support associative arrays.
- Pipe-alternation characters (`\|`) embedded in associative array **keys** are treated as literal strings during key iteration (`${!FRAMEWORKS[@]}`), not as grep regex alternation. The grep

command `grep -rq "GraceChain\|Gracechain"` works in a direct `run:` step, but when the key is extracted from the array and passed to grep, the escaping breaks.
- The `${#FRAMEWORKS[@]}` count and `${!FRAMEWORKS[@]}` iteration are bash-specific and produce syntax errors in POSIX sh.

Problem 2: No Physical Resonance Infrastructure

The deeper issue: the ERES Resonance Check is designed to validate **semantic coherence** across the ERES document corpus — but it currently operates as a static text-search (grep) against whatever `.md` files happen to exist in the repository at push time. This is a structural mismatch:

- **What the check should verify:** That all 16+ core ERES frameworks are referenced, correctly expanded, internally consistent, and semantically coherent across documents.
- **What the check actually does:** Runs grep patterns against file contents and counts matches.
- **What's missing:** A **resonance simulator** — a purpose-built diagnostic tool that can validate ERES terminology consistency, cross-reference definitions against ERES TERMS #46, detect acronym drift, and produce a structured coherence report.

The grep-based approach worked as a proof-of-concept, but the ERES framework corpus is now large enough (ERES TERMS #46 alone has 272 paragraphs, 22 sections, 150+ defined terms) that it needs a proper validation instrument.

Impact Assessment

Component	Status	Impact
Job 1: Validate	✅ PASS	SECURITY.md, README.md, CCAL present
Job 2: Integrity	✅ PASS	Principles, TERMS version, IPIDITIS, NPR all verified
Job 3: Resonance (Step 1)	✅ PASS	Locked acronyms (RHC, SOMT, REAL) clean
Job 3: Resonance (Step 2)	🔴 FAIL	Framework coverage report — bash infrastructure
Job 4: NPR Report	✅ PASS	Runs regardless (`if: always()`)

Net effect: CI workflow reports failure, but the failure is infrastructure-related, not content-related. The repository documents are consistent; the checker is broken.

Proposed Solution: ERES Resonance Simulator

Architecture

Replace the bash associative array grep approach with a standalone **Node.js resonance simulator** (`eres-resonance-check.js`) that:

1. **Loads the canonical ERES TERMS #46 reference** as a structured data source (JSON manifest)
2. **Scans all `.md` files** in the repository
3. **Validates locked acronyms** — checks every expansion of RHC, SOMT, CyberRAVE, GunnySack, SaleBuilders, REAL formula against canonical forms
4. **Measures framework coverage** — which of the 16+ core frameworks appear in the document corpus
5. **Detects acronym drift** — finds instances where an acronym is expanded differently than the canonical ERES TERMS definition
6. **Reports semantic coherence** — a resonance score (0–100%) based on coverage, consistency, and cross-reference density
7. **Outputs machine-readable results** — JSON for CI consumption, human-readable table for the log

File Structure

```
.github/
├── workflows/
|   └── ci.yml                    # Updated workflow (calls simulator)
└── eres-resonance/
    ├── eres-resonance-check.js   # Simulator engine
    ├── eres-terms-manifest.json  # Canonical terms from ERES TERMS #46
    └── README.md                 # Simulator documentation
```

Simulator Output Format

```
╔═══════════════════════════════════════════════════════════╗
||  ERES RESONANCE CHECK — Simulator v1.0                  ||
||  Reference: ERES TERMS #46 (March 28, 2026)             ||
╟═══════════════════════════════════════════════════════════╢
||                                                         ||
||  LOCKED ACRONYMS           ✓ 6/6 consistent             ||
||  FRAMEWORK COVERAGE        ✓ 16/16 referenced           ||
```

```
||  ACRONYM DRIFT              ✓ 0 inconsistencies        ||
||  CROSS-REFERENCES           ✓ 42 valid links           ||
||                                                       ||
||  RESONANCE SCORE: 98%                                 ||
||                                                       ||
||  C = R × P / M                                       ||
╚═══════════════════════════════════════════════════════╝
```

Implementation Tasks

- [ ] **Task 1:** Create `eres-terms-manifest.json` — structured JSON export of all ERES TERMS #46 entries with canonical expansions, locked status, and cross-references
- [ ] **Task 2:** Create `eres-resonance-check.js` — Node.js simulator that loads manifest, scans `.md` files, validates consistency, outputs resonance score
- [ ] **Task 3:** Update `ci.yml` Job 3 — replace bash associative array with `node .github/eres-resonance/eres-resonance-check.js` call
- [ ] **Task 4:** Add `eres-resonance/README.md` — document the simulator for contributors
- [ ] **Task 5:** Validate simulator against all five repo governance files (README.md, SECURITY.md, CODE_OF_CONDUCT.md, CONTRIBUTING.md, ERES_TERMS_46.md)

Acceptance Criteria

1. `eres-resonance-check.js` runs successfully on `ubuntu-latest` GitHub Actions runner
2. All 6 locked acronyms validated against canonical SPT Papers expansions
3. All 16+ core frameworks checked for presence in document corpus
4. Resonance score output is deterministic and reproducible
5. Zero false positives on the current governance document set
6. CI workflow passes end-to-end (all 4 jobs green)

NPR Classification

This issue is a **system learning opportunity**, not a failure of any contributor. The original bash implementation was a valid proof-of-concept that revealed the need for a proper resonance validation instrument. The simulator is the system's response to its own limitation.

Principle served: Build for generations to come — the simulator will scale with the ERES corpus as it grows toward 1000+ documents.

Labels

`infrastructure` · `ci` · `resonance` · `simulator` · `NPR-level-1`

Related Documents

- SECURITY.md — IPIDITIS Roadmap and GAIA SOMT 1000-Year Future Map
- CONTRIBUTING.md — Locked acronym table and terminology conventions
- CODE_OF_CONDUCT.md — NPR v3.0 remediation framework
- ERES TERMS #46 — Canonical terminology reference
- SPT Papers (Sprute, 2026) — Source of locked acronym expansions

---

"Don't hurt yourself. Don't hurt others. Build for generations to come."

CARE Commons Attribution License v2.1 (CCAL)